

DOO

Asana Connector

Technical Documentation · v1.0.0

Built by Idrees Khaled	Organization Doo
Email Idreesalamari@gmail.com	Phone 32255886
Stack TypeScript · Node.js · Express · React · Zod · MCP	Status Production-oriented · Local-first

Executive summary

A reusable, production-oriented Asana integration designed for the DOO Builders League. The connector exposes 35 typed actions across Tasks, Projects, Sections, Comments, Tags, and Users. A shared connector core powers both the web console and the MCP adapter, keeping validation, safety, rate limiting, error handling, and Asana communication in one source of truth.

The project is deliberately local-first: it runs with `npm run dev` and does not require a public deployment. The MCP adapter is implemented, while a public HTTPS MCP endpoint remains an optional future deployment step.

1. Project Overview

The Asana Connector acts as a reliable bridge between applications or AI agents and the Asana API. Instead of implementing business logic separately for the web console and MCP, both adapters call the same connector core.

Architecture

React Console → Express API → Connector Core → Asana Client → Asana API
MCP Adapter → Connector Core → Asana Client → Asana API

The core owns typed schemas, validation, normalized errors, rate limiting, and write-safety metadata. This prevents duplicated logic and keeps API and MCP behavior consistent.

Key capabilities

- **35 typed actions** covering the most useful Asana task and project workflows.
- **Real Asana API integration** with live verification of the required action cycle.
- **PAT authentication** for direct/local use and verified live access.
- **OAuth 2.0 with PKCE** for multi-user authentication; authorization was verified against Asana's live endpoint.
- **MCP support** through the official Model Context Protocol SDK, using a thin adapter over the shared core.
- **Write safety** with explicit approval metadata and conservative retry behavior.
- **Demo/local console** built with React 19, Vite, Tailwind CSS 4, TanStack Query, and Radix UI primitives.
- **263 automated tests**, clean typecheck/lint/secret scan, and CI across Node 20/22/24.

2. Technology Stack

Layer	Technology	Purpose
Backend	TypeScript (strict), Node.js, Express	HTTP API, application services, integration runtime
Validation	Zod	Typed input/output schemas and runtime validation
Frontend	React 19, Vite, Tailwind CSS 4	Premium local console and action playground
Data fetching	TanStack Query	Frontend server-state and request management
UI primitives	Radix UI	Accessible composable UI components
MCP	@modelcontextprotocol/sdk	AI-agent adapter and MCP tooling
Testing	Vitest	263 backend/frontend tests
Auth	PAT + OAuth 2.0 / PKCE	Direct authentication and multi-user authorization

3. Action Reference — 35 Actions

The five assignment-required actions remain unchanged and first in the registry. The remaining 30 actions extend the same architecture. Every action uses typed schemas, the real Asana client, normalized errors, rate limiting, and safety metadata.

Tasks (15)

Action	What it does
<code>asana.list_project_tasks</code>	List tasks belonging to a specific project.
<code>asana.create_task</code>	Create a new Asana task with supported task fields.
<code>asana.update_task</code>	Update an existing task using a controlled patch.
<code>asana.get_task</code>	Retrieve detailed information for a specific task.
<code>asana.list_tasks</code>	List tasks using supported Asana filters and pagination.
<code>asana.search_tasks</code>	Search tasks using Asana search capabilities; premium availability and pagination limitations are documented.
<code>asana.complete_task</code>	Mark a task as completed.
<code>asana.reopen_task</code>	Reopen a completed task.
<code>asana.assign_task</code>	Assign a task to a user.
<code>asana.set_task_due_date</code>	Set or update a task due date.
<code>asana.set_task_description</code>	Set or update the task description/notes.
<code>asana.create_subtask</code>	Create a subtask under a parent task.
<code>asana.list_subtasks</code>	List subtasks belonging to a task.
<code>asana.add_task_to_project</code>	Associate a task with a project.
<code>asana.remove_task_from_project</code>	Remove a task's project association without deleting the task.

Projects (7)

Action	What it does
<code>asana.list_projects</code>	List accessible Asana projects.
<code>asana.get_project</code>	Retrieve project details.
<code>asana.create_project</code>	Create a project.
<code>asana.update_project</code>	Update project metadata.
<code>asana.list_project_members</code>	List members associated with a project.
<code>asana.add_project_member</code>	Add a user to a project membership.
<code>asana.remove_project_member</code>	Remove a user's project membership without deleting the user or project.

Sections (4)

Action	What it does
asana.list_project_sections	List sections within a project.
asana.create_section	Create a new project section.
asana.update_section	Update section properties.
asana.move_task_to_section	Move a task into a project section.

Comments (2)

Action	What it does
asana.add_comment	Add a comment/story to a task.
asana.list_comments	List comments/stories associated with a task.

Tags (4)

Action	What it does
asana.list_tags	List accessible tags.
asana.create_tag	Create a tag.
asana.add_tag_to_task	Associate a tag with a task.
asana.remove_tag_from_task	Remove a tag association from a task without deleting the tag.

Users (3)

Action	What it does
asana.get_current_user	Retrieve the authenticated user's Asana profile.
asana.get_user	Retrieve a specific user's profile.
asana.list_users	List accessible users.

4. Authentication & Security

Personal Access Token (PAT)

PAT is the fastest authentication path for local development and live verification. Secrets are kept in the private, gitignored `.env` file and are never intended to be committed.

OAuth 2.0 + PKCE

OAuth is implemented for multi-user scenarios. The connector uses the configured client ID, client secret, redirect URI, and least-privilege scopes. The live Asana authorization endpoint accepted the application's client ID, PKCE parameters, and requested scopes. Interactive user consent was intentionally not performed by the developer because it requires the user's credentials.

Credential hygiene

- Real credentials are stored in `.env`, not `.env.example`.
- The tracked template contains placeholders only.
- Secrets are excluded from Git through gitignore and secret scanning.
- Credentials exposed during development should be rotated immediately.
- No credentials are printed to logs or returned through action responses.

5. Write Safety & Idempotency

There are **21 write actions**. Every write requires explicit approval metadata so a UI or AI agent can distinguish read-only operations from mutations.

Six create operations are never automatically retried: `create_task`, `create_project`, `create_section`, `create_subtask`, `create_tag`, and `add_comment`. Asana does not provide server-side idempotency for these operations, so a timeout could mean the object was created successfully. Retrying blindly could therefore create duplicates.

- Update-style mutations are treated as retryable only when repeating the same patch is safe.
- Association operations are designed to be safe to repeat where the resulting state is unchanged.
- There are zero delete actions.
- There are zero delete OAuth scopes.
- Safety behavior is represented in shared metadata so the same policy applies to API and MCP consumers.

6. Error Handling & Reliability

The connector normalizes Asana and transport failures into stable application-level error categories. Validation occurs before the request is sent, and rate limiting is enforced client-side to reduce avoidable pressure on the Asana API.

Pagination is implemented where Asana exposes cursor-based pagination. Where an Asana endpoint has known limitations, the connector documents those constraints instead of pretending the endpoint has capabilities it does not have.

7. Console / Frontend

The local console is a premium dark-first/modern interface for exploring and demonstrating the connector. It is built with React 19 and Vite, styled with Tailwind CSS 4, and uses TanStack Query plus Radix UI primitives.

- **Action Center:** all 35 actions are grouped by domain and searchable.
- **Schema Inspector:** exposes the current action schema rather than a hardcoded list.
- **Playground:** provides an interactive surface for running actions.
- **Activity:** filters and views use the live action registry.
- **Authentication:** supports the configured PAT/OAuth paths.
- **Validation and error states:** surface connector responses cleanly.
- **Accessibility:** color contrast was checked against WCAG; a slate-on-lavender badge contrast issue was found and fixed.

Visual system

Role	Color
Background	#FAF8FC — soft off-white with lavender hint
Main heading	#0F172A — dark charcoal
Heading highlight	#6B21A8 — dark purple
Body text	#64748B — slate gray
Primary / numbers	#7C3AED — vibrant purple
Badges / tags	#F3E8FF — light lavender

8. Testing & Verification

263 automated tests are currently reported: 238 backend tests and 25 frontend tests.

- TypeScript typecheck: clean.
- ESLint: clean.
- Secret scan: clean.
- Frontend build: successful.
- CI: green on Node 20, Node 22, and Node 24.
- PAT authentication: verified live.
- All five required actions: verified against the real Asana account in a complete read/write cycle.
- OAuth authorize step: verified against Asana's real endpoint; PKCE and configured scopes were accepted.
- `get_current_user`: verified against the real account.
- Demo/local console: built against the shared connector architecture.

9. Known Limitations — documented, not hidden

- **search_tasks**: Asana availability is premium-only and pagination behavior is not treated as universally stable.
- **list_tasks**: Asana requires a specific filter combination for certain requests; the connector validates this rather than surfacing a cryptic upstream error.
- **Project memberships**: the relevant endpoint is technically deprecated, but it is retained because the documented replacement does not clearly expose the required OAuth scope.
- **OAuth interactive consent**: the developer does not perform the user's login/consent step. The authorization endpoint itself was verified live.
- **Public HTTPS MCP endpoint**: not deployed yet by design; the project is local-first. The MCP adapter is implemented.
- **Delete**: intentionally unsupported. This is a safety/product decision, not an incomplete CRUD implementation.

10. Local Setup & Demo

- Clone the repository.
- Run `npm install`.
- Create the private environment file from the provided template.
- Configure PAT and/or OAuth values as required.
- Run `npm run dev`.
- Use the console to browse the 35-action registry, inspect schemas, and run supported actions.
- For live Asana verification, use a valid credential and clean up temporary test data manually when required.

Demo narrative

A strong demonstration path is: open the Action Center → show the 35 live actions → inspect a schema → authenticate → read the current user/projects → create or update a safe demo task with explicit approval → add a comment → show the resulting Asana state → explain the write-safety policy and shared MCP/API architecture.

11. Engineering Decisions

Single-source connector core

The most important architectural decision is keeping business and integration logic inside one core. The Express API and MCP adapter are intentionally thin. This reduces drift, simplifies testing, and ensures an AI agent and a human using the console receive the same validation and safety behavior.

Real-client demo path

Demo behavior is designed around the same client/action/validation/error pipeline rather than a completely separate feature implementation. This makes the console a useful review surface for the shipping architecture.

Least privilege

OAuth scopes are limited to the operations actually required. Delete scopes are intentionally absent because the connector does not expose deletion.

Honest compatibility

The documentation distinguishes between live-verified behavior and implemented-but-not-live-verified behavior. This is important for an integration project because an API contract can differ from an in-memory test implementation.

12. Submission Snapshot

Item	Status
Connector version	v1.0.0
Actions	35 total
Required actions	5 — preserved unchanged
Extended actions	30
Authentication	PAT + OAuth 2.0 / PKCE
MCP	Implemented; local-first; public HTTPS endpoint not deployed
Tests	263
CI	Green — Node 20/22/24
Delete operations	None
Primary author	Idrees Khaled
Organization	Doo

Prepared for DOO Builders League

Prepared by **Idrees Khaled** · Idreesalamari@gmail.com · 32255886

This document describes the implemented v1.0.0 connector architecture, action surface, security model, testing status, and known limitations based on the project status provided for documentation.